

TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA CÔNG NGHỆ THÔNG TIN

**ĐỀ THI GIỮA HỌC KỲ VÀ BÀI LÀM**

Tên học phần: Toán ứng dụng CNTT

Mã học phần: Hình thức thi: *Tự luận*

Đề số: **03** Thời gian làm bài: 90 phút (*không kể thời gian chép/phát đề*)

Được sử dụng tài liệu khi làm bài.

**Họ tên:** .....Lê Thị Diệu Hòa.....**Lớp:**.....24T\_KHDL.....**MSSV:**.....102240244.....

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV\_HọTên.pdf và nộp bài thông qua MSTeam

**Câu 1:** (3 điểm) Cho số nguyên dương  $N=12000$ . Viết chương trình bằng C/C++ có sử dụng hàm thực hiện:

- Tìm số lượng các số hoàn hảo nhỏ hơn  $N$ , liệt kê và tính tổng của chúng.
- Gọi  $M$  là số hoàn hảo lớn nhất vừa tìm được, tìm số  $K$  gần  $M$  nhất sao cho  $K$  chia hết cho 7.
- Tìm số lượng các số nguyên dương nhỏ hơn  $K$  và nguyên tố cùng nhau với  $K$ . Liệt kê các số đó và tính tổng của chúng.
- Tìm số  $P$  là số nguyên tố gần  $K$  nhất

**# Trả lời:** Dán code vào bên dưới:

```
#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

long long N = 0;

long long gcd(long long a, long long b)
{
    if (b == 0)
        return a;
    return gcd(b, a % b);
}

bool isPrime(long long n)
{
    if (n < 2)
        return false;
    for (long long i = 2; i * i <= n; i++)
        if (n % i == 0)
            return false;
    return true;
}

bool isPerfectNumber(long long n)
{
    if (n < 2)
        return false;
    long long sum = 1;
    for (long long i = 2; i * i <= n; i++)
```

```

    {
        if (n % i == 0)
        {
            sum += i;
            if (i != n / i)
                sum += n / i;
        }
    }
    return sum == n;
}

vector<long long> perfectNumbersLessThan(long long M)
{
    vector<long long> list;
    for (long long i = 2; i < M; i++)
        if (isPerfectNumber(i))
            list.push_back(i);
    return list;
}

long long nearestDivisibleBy7(long long K)
{
    if (K % 7 == 0)
        return K;
    long long below = K - (K % 7);
    long long above = below + 7;
    return (K - below) <= (above - K) ? below : above;
}

vector<long long> coprimes(long long K)
{
    vector<long long> list;
    for (long long i = 1; i < K; i++)
        if (gcd(i, K) == 1)
            list.push_back(i);
    return list;
}

long long nearestPrime(long long K)
{
    if (isPrime(K))
        return K;
    for (long long d = 1;; d++)
    {
        if (K - d >= 2 && isPrime(K - d))
            return K - d;
        if (isPrime(K + d))
            return K + d;
    }
}

int main()
{
    do
    {
        cout << "Nhap so nguyen duong N: ";
        cin >> N;
    } while (N <= 0);

    vector<long long> perfects = perfectNumbersLessThan(N);

```

```

long long sum = 0;
cout << "1. Cac so hoan hao nho hon N = " << N << ":" << endl;
if (perfects.empty())
{
    cout << "    Khong co so hoan hao nao" << endl;
}
else
{
    cout << "    So luong: " << perfects.size() << endl;
    cout << "    Danh sach: ";
    for (long long i = 0; i < (long long)perfects.size(); i++)
    {
        cout << perfects[i];
        if (i < (long long)perfects.size() - 1)
            cout << ", ";
        sum += perfects[i];
    }
    cout << endl;
    cout << "    Tong: " << sum << endl;
}
cout << endl;

long long M = perfects.back();
long long K = nearestDivisibleBy7(M);
cout << "2. So hoan hao lon nhat: M = " << M << endl;
cout << "    So K gan M nhat va chia het cho 7: K = " << K << endl;
cout << endl;

vector<long long> coprimeList = coprimes(K);
sum = 0;
cout << "3. Cac so nguyen duong nho hon K = " << K << " va nguyen to cung nhau
voi K:" << endl;
cout << "    So luong: " << coprimeList.size() << endl;
cout << "    Danh sach: ";
for (long long i = 0; i < (long long)coprimeList.size(); i++)
{
    cout << coprimeList[i];
    if (i < (long long)coprimeList.size() - 1)
        cout << ", ";
    sum += coprimeList[i];
}
cout << endl;
cout << "    Tong: " << sum << endl;
cout << endl;

long long P = nearestPrime(K);
cout << "4. So nguyen to P gan K = " << K << " nhat: P = " << P << endl;
cout << "    (Khoang cach: |" << K << " - " << P << "| = " << abs(K - P) << ")"
<< endl;

return 0;
}

```

**Kết quả thực thi:**



```

    if (n == 0)
        return 1 % m;
    long long u = modpow(x, n / 2, m);
    u = (u * u) % m;
    if (n % 2 == 1)
        u = (u * x) % m;
    return u;
}

long long eulerPhi(long long n)
{
    long long result = n;
    for (int i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
        {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}

vector<long long> modInverse(vector<long long> &x, vector<long long> &m)
{
    vector<long long> x_inverse;
    for (int i = 0; i < x.size(); i++)
    {
        x_inverse.push_back(modpow(x[i], eulerPhi(m[i]) - 1, m[i]));
    }
    return x_inverse;
}

int main()
{
    long long n;
    cout << "Nhap so phuong trinh: ";
    cin >> n;
    vector<long long> a;
    vector<long long> m;
    long long num, rem;

    for (long long i = 1; i <= n; i++)
    {
        cout << "Nhap a[" << i << "], m[" << i << "]: ";
        cin >> num >> rem;
        a.push_back(num);
        m.push_back(rem);
    }

    vector<long long> X;
    long long product = 1;
    for (int i = 0; i < n; i++)
        product *= m[i];
    for (int i = 0; i < n; i++)
        X.push_back(product / m[i]);
}

```

```

vector<long long> X_inverse = modInverse(X, m);

long long x = 0;
for (int i = 0; i < n; i++)
    x += a[i] * X[i] * X_inverse[i];
cout << "Nghiem tong quat: " << x % product << " + k" << product << endl;
return 0;
}

```

**# Trả lời:** Dán kết quả thực thi vào bên dưới:

```

Nhap so phuong trinh: 5
Nhap a[1], m[1]: 2 5
Nhap a[2], m[2]: 4 7
Nhap a[3], m[3]: 6 9
Nhap a[4], m[4]: 9 13
Nhap a[5], m[5]: 12 17
Nghiem tong quat: 9447 + k69615

```

**Câu 3: (5 điểm)** Cho ma trận

$$A = \begin{bmatrix} 4 & 1 & 2 \\ 1 & 8 & 2 \\ 2 & 2 & 10 \end{bmatrix}.$$

Viết chương trình bằng c/c++ có sử dụng hàm thực hiện phân rã ma trận A.

a) Phân rã  $LDL^T$  ma trận A

**# Trả lời:** Dán code vào bên dưới (bao gồm điều kiện của ma trận A nếu có):

Điều kiện:

- Ma trận phải là ma trận vuông và đối xứng

```

#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>

using namespace std;

#define EPS 1e-5
using Matrix = vector<vector<double>>>;

void printMatrix(const Matrix &A)
{
    for (int i = 0; i < (int)A.size(); i++)
    {
        for (int j = 0; j < (int)A[i].size(); j++)
            cout << fixed << setprecision(6) << setw(10) << A[i][j];
        cout << endl;
    }
    cout << endl;
}

Matrix multiply(const Matrix &A, const Matrix &B)

```

```

{
    int m = A.size(), n = B[0].size(), p = B.size();
    Matrix C(m, vector<double>(n, 0.0));
    for (int i = 0; i < m; i++)
        for (int k = 0; k < p; k++)
            for (int j = 0; j < n; j++)
                C[i][j] += A[i][k] * B[k][j];
    return C;
}

```

Matrix transpose(const Matrix &A)

```

{
    int m = A.size(), n = A[0].size();
    Matrix T(n, vector<double>(m));
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            T[j][i] = A[i][j];
    return T;
}

```

bool isSquare(const Matrix &A)

```

{
    return (int)A.size() == (int)A[0].size();
}

```

bool isSymmetric(const Matrix &A)

```

{
    int n = A.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (fabs(A[i][j] - A[j][i]) > EPS)
                return false;
    return true;
}

```

double det(Matrix A)

```

{
    int n = A.size();
    double result = 1.0;
    for (int i = 0; i < n; i++)
    {
        if (fabs(A[i][i]) < EPS)
        {
            bool found = false;
            for (int j = i + 1; j < n; j++)
            {
                if (fabs(A[j][i]) > EPS)
                {
                    swap(A[i], A[j]);
                    result *= -1;
                    found = true;
                    break;
                }
            }
            if (!found)
                return 0.0;
        }
        result *= A[i][i];
        for (int j = i + 1; j < n; j++)
        {

```

```

        double ratio = A[j][i] / A[i][i];
        for (int k = i; k < n; k++)
            A[j][k] -= ratio * A[i][k];
    }
}
return result;
}

bool isPositiveDefinite(const Matrix &A)
{
    int n = A.size();
    for (int k = 1; k <= n; k++)
    {
        Matrix sub(k, vector<double>(k));
        for (int i = 0; i < k; i++)
            for (int j = 0; j < k; j++)
                sub[i][j] = A[i][j];
        if (det(sub) <= EPS)
            return false;
    }
    return true;
}

void ldlt(const Matrix &A)
{
    int n = A.size();

    Matrix L(n, vector<double>(n, 0.0));
    vector<double> d(n, 0.0);
    for (int i = 0; i < n; i++)
        L[i][i] = 1.0;

    for (int j = 0; j < n; j++)
    {
        double sum_d = 0.0;
        for (int k = 0; k < j; k++)
            sum_d += L[j][k] * L[j][k] * d[k];
        d[j] = A[j][j] - sum_d;

        if (fabs(d[j]) < EPS)
        {
            cout << "  d[" << j << "] = 0, "
                  << "ma tran suy bien, khong the phan ra LDL^T" << endl;
            return;
        }

        for (int i = j + 1; i < n; i++)
        {
            double sum_l = 0.0;
            for (int k = 0; k < j; k++)
                sum_l += L[i][k] * L[j][k] * d[k];
            L[i][j] = (A[i][j] - sum_l) / d[j];
        }
    }

    Matrix D(n, vector<double>(n, 0.0));
    for (int i = 0; i < n; i++)
        D[i][i] = d[i];

    Matrix LT = transpose(L);

```



```

Matrix check = multiply(multiply(L, D), LT);

cout << "Ma tran L:" << endl;
printMatrix(L);

cout << "Ma tran D:" << endl;
printMatrix(D);

cout << "Ma tran LT:" << endl;
printMatrix(LT);

cout << "Ket qua L * D * LT:" << endl;
printMatrix(check);
}

int main()
{
    int n;
    cout << "Nhap kích thước ma tran: ";
    cin >> n;

    Matrix A(n, vector<double>(n));
    cout << "Nhap ma tran A:" << endl;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> A[i][j];

    cout << "Ma tran A:" << endl;
    printMatrix(A);

    if (!isSquare(A))
    {
        cout << "Ma tran không phải ma tran vuông!" << endl;
        return 1;
    }
    if (!isSymmetric(A))
    {
        cout << "Ma tran không đối xứng -> không thể phân ra LDL^T!" << endl;
        return 1;
    }

    ldlt(A);

    return 0;
}

```

**# Trả lời:** Dán kết quả thực thi vào bên dưới (sai số  $\varepsilon = 10^{-5}$ ):

```

Nhap kích thước ma tran: 3
Nhap ma tran A:
4 1 2
1 8 2
2 2 10
Ma tran A:
4.000000 1.000000 2.000000
1.000000 8.000000 2.000000
2.000000 2.000000 10.000000

Ma tran L:
1.000000 0.000000 0.000000
0.250000 1.000000 0.000000
0.500000 0.193548 1.000000

Ma tran D:
4.000000 0.000000 0.000000
0.000000 7.750000 0.000000
0.000000 0.000000 8.709677

Ma tran LT:
1.000000 0.250000 0.500000
0.000000 1.000000 0.193548
0.000000 0.000000 1.000000

Ket qua L * D * LT:
4.000000 1.000000 2.000000
1.000000 8.000000 2.000000
2.000000 2.000000 10.000000

```

b) Phân rã **eigendecomposition** ma trận A

# **Trả lời:** Dán code vào bên dưới (bao gồm điều kiện của ma trận A nếu có):

Điều kiện:

- Ma trận phải là ma trận vuông.
- Phải có đủ các giá trị riêng và vector riêng.

```

#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <algorithm>
#include <numeric>
#include "../eigen/Eigen/Dense"

using namespace std;

#define EPS 1e-5
using Matrix = vector<vector<double>>>;

void printMatrix(const Matrix &A)
{
    for (int i = 0; i < (int)A.size(); i++)
    {
        for (int j = 0; j < (int)A[i].size(); j++)
            cout << fixed << setprecision(6) << setw(12) << A[i][j];
        cout << endl;
    }
    cout << endl;
}

```

```

}

Matrix multiply(const Matrix &A, const Matrix &B)
{
    int m = A.size(), n = B[0].size(), p = B.size();
    Matrix C(m, vector<double>(n, 0.0));
    for (int i = 0; i < m; i++)
        for (int k = 0; k < p; k++)
            for (int j = 0; j < n; j++)
                C[i][j] += A[i][k] * B[k][j];
    return C;
}

Matrix transpose(const Matrix &A)
{
    int m = A.size(), n = A[0].size();
    Matrix T(n, vector<double>(m));
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            T[j][i] = A[i][j];
    return T;
}

double det(Matrix A)
{
    int n = A.size();
    double result = 1.0;
    for (int i = 0; i < n; i++)
    {
        if (fabs(A[i][i]) < EPS)
        {
            bool found = false;
            for (int j = i + 1; j < n; j++)
                if (fabs(A[j][i]) > EPS)
                {
                    swap(A[i], A[j]);
                    result *= -1;
                    found = true;
                    break;
                }
            if (!found)
                return 0.0;
        }
        result *= A[i][i];
        for (int j = i + 1; j < n; j++)
        {
            double ratio = A[j][i] / A[i][i];
            for (int k = i; k < n; k++)
                A[j][k] -= ratio * A[i][k];
        }
    }
    return result;
}

void computeEigen(const Matrix &A, vector<double> &eigenvalues, Matrix
&eigenvectors)
{
    int n = A.size();
    Eigen::MatrixXd Aeig(n, n);
    for (int i = 0; i < n; i++)

```

```

        for (int j = 0; j < n; j++)
            Aeig(i, j) = A[i][j];

Eigen::SelfAdjointEigenSolver<Eigen::MatrixXd> solver(Aeig);

eigenvalues.resize(n);
eigenvectors.assign(n, vector<double>(n));
for (int i = 0; i < n; i++)
{
    eigenvalues[i] = solver.eigenvalues()(i);
    for (int j = 0; j < n; j++)
        eigenvectors[j][i] = solver.eigenvectors()(j, i);
}
}

void eigenDecomposition(const Matrix &A)
{
    int n = A.size();

    vector<double> eigenvalues;
    Matrix P;
    computeEigen(A, eigenvalues, P);

    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(), [&](int a, int b)
        { return eigenvalues[a] > eigenvalues[b]; });

    vector<double> ev(n);
    Matrix P_sorted(n, vector<double>(n));
    for (int j = 0; j < n; j++)
    {
        ev[j] = eigenvalues[idx[j]];
        for (int i = 0; i < n; i++)
            P_sorted[i][j] = P[i][idx[j]];
    }

    Matrix D(n, vector<double>(n, 0.0));
    for (int i = 0; i < n; i++)
        D[i][i] = ev[i];

    Matrix Pinv = transpose(P_sorted);

    if (fabs(det(P_sorted)) < EPS)
    {
        cout << "P không khả nghịch => Ma trận không chéo hóa được!" << endl;
        return;
    }

    cout << "Tri riêng:" << endl;
    for (int i = 0; i < n; i++)
        cout << "  lambda[" << i + 1 << "] = " << fixed << setprecision(6) << ev[i]
<< endl;
    cout << endl;

    cout << "Vectơ riêng:" << endl;
    printMatrix(P_sorted);

    cout << "P:" << endl;
    printMatrix(P_sorted);
}

```

```

    cout << "D:" << endl;
    printMatrix(D);

    cout << "P^T:" << endl;
    printMatrix(Pinv);

    cout << "P * D * P^T:" << endl;
    printMatrix(multiply(multiply(P_sorted, D), Pinv));
}

int main()
{
    int n;
    cout << "Nhap kích thước ma tran: ";
    cin >> n;

    Matrix A(n, vector<double>(n));
    cout << "Nhap ma tran A:" << endl;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> A[i][j];

    cout << "\nMa tran A:" << endl;
    printMatrix(A);

    eigenDecomposition(A);

    return 0;
}

```

**# Trả lời:** Dán kết quả thực thi vào bên dưới (sai số  $\varepsilon = 10^{-5}$ ):

```

Nhap kích thước ma tran: 3
Nhap ma tran A:
4 1 2
1 8 2
2 2 10

Ma tran A:
4.000000  1.000000  2.000000
1.000000  8.000000  2.000000
2.000000  2.000000  10.000000

Tri riêng:
lambda[1] = 11.867132
lambda[2] = 6.774309
lambda[3] = 3.358559

Vecto riêng:
-0.272615  0.051591  0.960739
-0.496661 -0.862774 -0.094600
-0.824020  0.502951 -0.260829

P:
-0.272615  0.051591  0.960739
-0.496661 -0.862774 -0.094600
-0.824020  0.502951 -0.260829

D:
11.867132  0.000000  0.000000
0.000000  6.774309  0.000000
0.000000  0.000000  3.358559

P^T:
-0.272615 -0.496661 -0.824020
0.051591 -0.862774  0.502951
0.960739 -0.094600 -0.260829

P * D * P^T:
4.000000  1.000000  2.000000
1.000000  8.000000  2.000000
2.000000  2.000000  10.000000

```